

Designing and Implementing Real-Time Bus Time Predictions using Artificial Intelligence

Benny Wai¹ and Winston Zhou¹

Transportation Research Record
1–13

© National Academy of Sciences:
Transportation Research Board 2020
Article reuse guidelines:

sagepub.com/journals-permissions

DOI: 10.1177/0361198120947715

journals.sagepub.com/home/trr



Abstract

Managing expectations is vital to ensuring commuter satisfaction with their public transportation service. When customers are given an estimated time for their bus and must wait way longer, they lose a sense of control over their commute and become frustrated. In this paper, we present a novel machine learning (ML) modeling approach in which we train and implement specialized models for every single segment of travel time and stop dwell time in our system to capture its uniqueness. The features for training the models include simple calendar and weather data. Most papers assume ideal operational conditions but in practice that's almost never the case. Here, we combine our ML modeling approach with a flexible production-tested algorithm to combine model-generated dwell and travel time (or run time) predictions to produce predicted bus departure times. This algorithm is designed to handle real transit agency challenges like missing models (including those caused by changes to schedules and routes), timing points, and partially traveled segments. We also provide a reference architecture of how this algorithm can be brought to life in a scalable and cost-effective manner.

TransLink is the statutory authority for the regional transportation network of Metro Vancouver. The various modes of transportation provided by TransLink include buses, SkyTrain, Seabus, and HandyDART (shared ride service for customers with physical or cognitive disabilities). In 2018, there were a total of 437 million annual system-wide boardings, 267 million of which were bus boardings.

One of the most common subjects of customer complaints is incorrect predicted bus departure times (PBDTs). The Real Time Transit Information (RTTI) is a TransLink-built system for processing and serving real-time bus locations and PBDTs to the public. Customers receive PBDTs through TransLink's NextBus website and third-party apps that consume data from our Open API, and General Transit Feed Specification (GTFS) feed. Before this project, PBDTs were estimated by shifting planned schedules (based on historical averages) by delays provided by real-time automated vehicle location (AVL) data. However, this is susceptible to changing weather and traffic conditions as these PBDTs don't take both pieces of information into account.

In 2018, TransLink attempted to resolve this issue using ML. After analyzing the available data, TransLink decided that with no real-time forecasted traffic data, a key determinant of travel time in Vancouver, an alternative was needed.

The next best alternative was the variable Time of Day which indirectly embeds traffic information. The problem with Time of Day is that it affects run time on each segment differently, so we decided to create one model per segment to capture that fact. Likewise, we don't have demand data to reliably predict dwell time for every stop using a single model, so we used Time of Day again. Since Time of Day also affects dwell time for different stops differently, we also decided to create one model for each stop to capture the difference. According to our literature review, capturing the differences this way has never been done before. If this novel approach works reasonably well, then we will save substantial costs involved in collecting and processing these unavailable data sets.

Since our models produce individual dwell and run-time predictions, we needed an algorithm to combine them to derive PBDTs. We call this the Aggregator Algorithm. For it to generate reliable PBDTs, it must also handle the following:

¹TransLink, Vancouver, BC, Canada

Corresponding Author:

Winston Zhou, Winston.Zhou@translink.ca

1. Partially traveled segments—our ML models for segments are trained to predict the run time of the entire segment but buses in transit are rarely right at the start or end of these segments at the time of prediction. Therefore, the algorithm must prorate the predicted run time based on the location of the bus within the segment.
2. Timing points—for most transit agencies, there are stops on each route designated as timing points, where bus operators who are early are instructed not to depart until the scheduled departure time. The algorithm must account for this rule for it to produce PBDTs accurately.
3. Missing models—models require enough training data to produce reliable predictions; however, training data may not be always available. This can be caused by service changes such as the introduction of new stops, or removal or displacement of existing stops. A new stop introduces a new dwell and can generate one or two new segments depending where it's positioned on the route. A removed stop can generate a new segment depending on its position on the route as well. In both cases there is no historical dwell and travel time data for the newly generated stops and segments. It is seldom a case for TransLink's bus network, but we do have stops and segments that are serviced so infrequently that there is not enough training data. In all these cases, the algorithm must be smart enough to use other non-ML predictions as substitutes when models are not available. If the Aggregator Algorithm is not capable of handling this case, then it can only project PBDTs until it runs into a missing model.

In this paper we will discuss how we used our granular modeling approach with our complementary Aggregator Algorithm to incorporate business rules and handle missing models to generate reliable PBDTs for our customers. We start by providing an understanding of the environment we operate in and the data sets available to us in real time. Next, we look at the features of our models and then finally how we tie everything together using our Aggregator Algorithm to produce PBDTs. The results are then analyzed.

Literature Review

With the emergence of ML, there has been a growing number of research studies on bus time predictions using ML algorithms such as artificial neural networks (ANN), Kalman filters, and regression models.

Amita et al. demonstrated that ANN was able to predict bus travel times better than regression models and

suggested that more research should be done to compare Kalman filtering to ANN (1). Fan and Gurmu demonstrated that ANN outperformed Kalman filtering models and used the historical average approach as a baseline to compare other methods against (2). Kumar et al. also demonstrated that ANN outperformed Kalman filters with large databases when predicting bus arrival times (3).

Yin et al. explained that while the lowest grain of data that their models used was the time between two consecutive bus stop arrival times, the components in this time were run time, halts at intersections, and dwell time at stops (4). Yin et al. mentioned that while their support vector machines (SVM) and ANN models to predict bus arrival time demonstrated value, it could be further improved by adding additional variables such as weather data (4).

Patnaik et al. showed that just by using automatic passenger counting (APC) data, you could build a multivariate linear regression model to predict bus arrival times (5). Shalaby and Farhan created a model that predicts both bus arrival and departure times using both APC and AVL data (6).

Peterson et al. showed a way to account for seasonal recurring variability, including traffic, by training on-time and calendar features for each link to generate weights (7). These weights are then fed to the link model to predict the link travel time.

Data

For this project, a variety of data sets were used both at prediction time and during model training. The service performance data set provides the actual arrival and departure times at stops for every historical bus event. The AVL data set provides real-time global positioning system (GPS) data for buses. This data is sent to servers whenever a bus leaves or passes by a bus stop, or every 90 s, whichever comes first. We also use the scheduled arrival/departure time for stops in every bus trip.

We used the Dark Sky API (8) to acquire real-time and forecasted weather data for the next 4 h. Weather depends on the location of the segment or bus stop, so we broke the bus transit system into 13 geographical regions as shown in Figure 1 and pulled data for each one.

Model

Let us define a segment as the path between two bus stops that exists in at least one line and pattern in the transportation system. We want to create two types of machine learning (ML) model sets: a set of models for the dwell time and a set models for the run time. Both

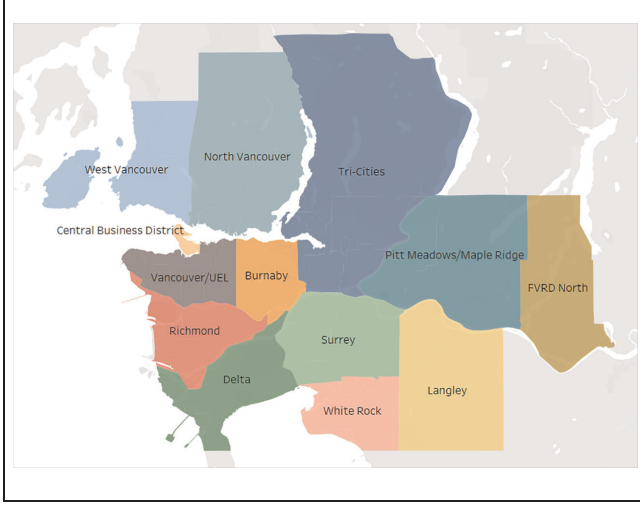


Figure 1. A map of the Vancouver Lower Mainland, separated by geographical regions used to pull weather data.

types of sets are identical except they are constructed to predict either the dwell time or run time, and the input data is adjusted to reflect that. Each set of models is made up of two ML models.

The dwell time model set predicts the time buses spend at a stop. The run time model set predicts the time buses spend traveling a segment. For each set of models, the first model takes all available information and comes up with a prediction of the offset between the scheduled and actual run or dwell time. The second model takes all the features used by the first model and adds the offset prediction value as a feature to come up with a run or dwell time prediction (Figure 2). Each bus stop and segment in the transit system has its own set of ML models, if there is enough data to train on (at least 500 instances of dwell/run events).

This approach ensures that the models are usable for multiple buses and lines, therefore minimizing the

number of models needed while ensuring that each model has enough training data.

The Aggregator Algorithm is model agnostic and will work with any ML model. In our case we chose eXtreme Gradient Boosting (XGBoost) because of its fast training time (approximately 4min per model if using one machine) and high-performance rate (as seen in many Kaggle competitions [9]) at the time of writing. Average run/dwell times are used as a baseline for comparison. As we create and train a different set of XGBoost models for every single segment and bus stop in our system, we end up with over 18,000 different sets of models (36,000 XGBoost models). A list of all the features, offsetDelta intermediate feature, and the objective output is seen in Table 1. XGBoost hyperparameters for each model were found using Grid Search (10).

Aggregator Algorithm

With the ML models creating predicted duration times for individual segments and stops (model-level predictions), we need to create an algorithm that aggregates these predictions together to create PBDTs. As the arrival/departure predictions for a stop are dependent on the prediction times for the stops before it, we use a recursive process to calculate predictions. Every time new information about a bus location or weather is received in our system, these bus-level predictions are recalculated using the latest relevant model-level predictions. Because of a logic rule in our AVL system, messages are only triggered when the buses are moving or leaving a bus stop.

To mathematically illustrate the algorithm, we first define all required variables in Table 2. For a bus of interest, the Aggregator Algorithm calculates the arrival time (Equation 3) for the immediate subsequent bus stop first. It then uses that information to calculate the subsequent departure time (Equation 4), and it continues this process downstream of the bus stop sequence in a

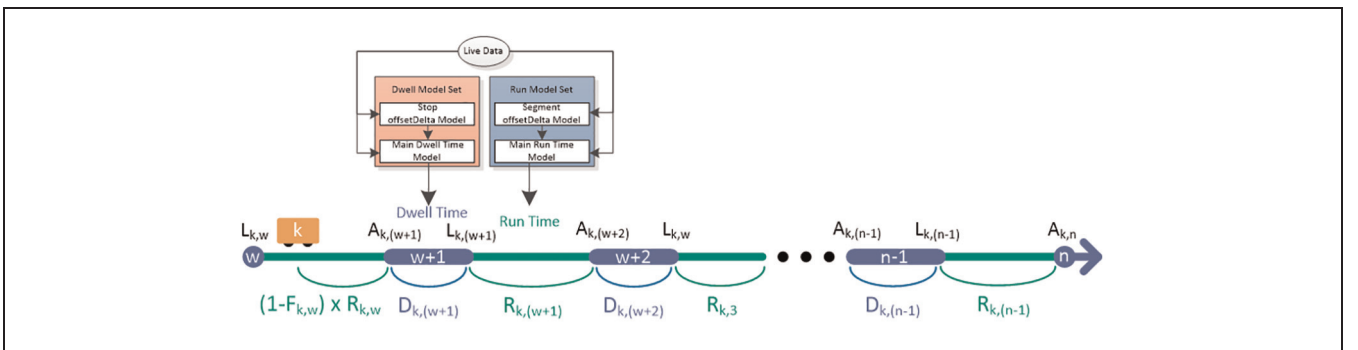


Figure 2. A diagram that shows the sets of dwell and run time models that use live data to produce model-level predictions. These model-level predictions are then aggregated to create the predicted departure time for stop n in this example. The model variables are explained in Table 2.

Table 1. Table of Model Input Variables

Variable/index	Description	Data type
ActMonth	Month	Integer
ActDayOfWeek	Day of the week	Integer
ActHourOfDay	Hour of the day	Integer
ActMinuteOfHour	Minute of the hour	Integer
ActDay	Day of month	Integer
VehCarryCapacity	Maximum suggested carrying capacity of the bus of interest	Integer
PrecipProbability	Chance of precipitation	Float
PrecipIntensityInMLsPerHour	Intensity of precipitation	Float
TemperatureInCelsius	Temperature in Celsius	Float
ApparentTemperatureInCelsius	Apparent temperature in Celsius	Float
DewPointInCelsius	Dew point in Celsius	Float
Humidity	Humidity	Float
PressureInHectopascals	Pressure	Float
WindSpeedInMetresPerSecond	Wind speed	Float
WindGustInMetresPerSecond	Wind bearing	Integer
CloudCover	Cloud cover	Binary
UVIndex	UV Index	Float
VisibilityInKM	Visibility in km	Float
ServiceTypeCode	Service type based on day type, categorical: MF, SAT, SUN, XMS, XBX, XEM	String
Icon	Weather icon, categorical: clear-day, clear-night, cloudy, fog, partly-cloudy-day, partly-cloudy-night, rain, sleet, snow, wind	String
SchTime	Scheduler's prediction of how long it will take for the bus to traverse the segment or dwell at a bus stop, in seconds	Integer
OffsetDelta	Prediction of the offset, in seconds, the run or dwell time is off by compared with schedule. A positive number means it is predicted to take less time than scheduled. For each set of dwell or run time models, this is the output of the first model to be fed into the second main prediction model as an additional feature.	Integer
RunTime/DwellTime	Prediction of the run or dwell time, in seconds. During training, this variable is the actual recorded time.	Integer

Note: All time and weather metrics are based on the predicted segment start time or stop arrival time, or the current time if the bus is on the segment of interest. ServiceTypeCode: MF = Monday to Friday; SAT = Saturday; SUN = Sunday; XMS = Christmas Day; XBX = Boxing Day; XEM = Easter Monday.

Table 2. Table of Variables

Variable/index	Description
k	Index for specific vehicle
s	Index for stop sequence for vehicle k , based on current line/pattern
w	Current stop sequence that vehicle k is on at time of message
$R_{k,s}$	Run time prediction for sth stop segment, based on line/pattern of vehicle k
$D_{k,s}$	Dwell time prediction for sth stop sequence, based on line/pattern of vehicle k
$A_{k,s}$	Predicted arrival time at stop sequence s for vehicle k , where $s > 1$
$L_{k,s}$	Predicted departure time at stop sequence s for vehicle k , where $s > 1$
$C_{k,s}$	Scheduled departure time for stop sequence s of vehicle k
V_k	Time of latest message for vehicle k
$F_{k,s}$	Fraction of stop segment s traversed at time of latest message for vehicle k
Z_k	Set of bus stops are that considered to be nodes ^{ϕ} for vehicle k
$MPR_{k,s}$	Model predicted run time for sth stop segment, based on line/pattern of vehicle k
$MAR_{k,s}$	Moving average run time for sth stop segment, based on line/pattern of vehicle k
$SR_{k,s}$	Scheduled run time for sth stop segment, based on line/pattern of vehicle k
$MPD_{k,s}$	Model predicted dwell for sth stop segment, based on line/pattern of vehicle k
$MAD_{k,s}$	Moving average dwell for sth stop segment, based on line/pattern of vehicle k
$SD_{k,s}$	Scheduled dwell for sth stop segment, based on line/pattern of vehicle k

Note: Nodes ^{ϕ} = bus stops with departure times that have timing point adjustments according to a schedule.

recursive process of period two. As previously mentioned, certain bus stops in a line are considered nodes, which are timing points to ensure the bus isn't too far ahead of the schedule. At these timing points, bus operators are instructed to leave at the scheduled time if they arrive early. The equation for the departure time takes this into account and uses scheduled departure time if the bus is early.

To handle missing models caused by insufficient training data (fewer than 500 historical events), the following precedence is applied to generate a dwell/runtime prediction.

1. If the model exists, use model prediction.
2. If the model does not exist, use moving average. This is useful as an intermediate backup when a stop is added or removed, and we've had some runs but not enough to generate a reliable model. In TransLink, the moving average is the average dwell/runtime based on the last two years of historical data aggregated by both the day of the week and the hour of the day. This can be changed as desired based on transit system requirements or changing conditions. For COVID-19, when traffic conditions eased and passenger load dropped significantly, TransLink was able to still provide accurate PBDTs by using a 2-week moving average and removing all models. When there is enough data for the models to train during this unique period, we can replace moving average with models again. This showcases the Aggregator Algorithm's ability to adapt when running into situations that are not reflected in the model training data.
3. If the model and moving average does not exist, use scheduled times. Scheduled times are used when a stop is added or removed, and we don't have data to generate the moving average yet.
 - a. Scheduled dwell time = scheduled departure time for a stop—the scheduled arrival time for the same stop.
 - b. Scheduled runtime = scheduled arrival time for end stop—scheduled departure time for start stop.

This is done to ensure that missing models for individual bus stops and segments won't prevent predictions from being made for the rest of the trip. This gives us the following equations:

$$\begin{aligned}
 R_{k,s} &= \text{MPR}_{k,s} \text{ if ML|model exists} \\
 &\text{MAR}_{k,s} \text{ if ML|model does not exist} \\
 &\text{SR}_{k,s} \text{ if ML model and Moving Average do not exist}
 \end{aligned}
 \tag{1}$$

$$\begin{aligned}
 D_{k,s} &= \text{MPD}_{k,s} \text{ if ML|model exists} \\
 &\text{MAD}_{k,s} \text{ if ML|model does not exist} \\
 &\text{SD}_{k,s} \text{ if ML model and Moving Average do not exist}
 \end{aligned}
 \tag{2}$$

Predictions are only made for bus stops that the bus hasn't reached yet, so for $s > w$

$$A_{k,s} = \begin{cases} V_k + (1 - F_{k,(s-1)}) \times R_{k,(s-1)} & \text{if } w = (s-1) \\ L_{k,(s-1)} + R_{k,(s-1)} & \text{if } w \neq (s-1) \end{cases}
 \tag{3}$$

$$L_{k,s} = \begin{cases} A_{k,s} + D_{k,s} & \text{if } s \notin Z_k \\ \max(C_{k,s}, A_{k,s} + D_{k,s}) & \text{if } s \in Z_k \end{cases}
 \tag{4}$$

Implementation Architecture

While the ML logic is established, the implementation architecture is constrained by many real-world requirements. RTTI is a real-time system; departure time predictions must be generated and served to customers in a timely manner, as predictions become stale in minutes. Moreover, the implementation method must be able to generate predictions every 90s for up to a total 1,200 buses and their upcoming stops. With each line and pattern having an average of 30 segments and 30 stops (an average remaining 30/2 segments and 30/2 stops), the system must be able to generate over 20,000 predictions per minute. In contrast to a system in which one model makes all the predictions, 20,000 predictions per minute using distinct models for each dwell and run time is a lot more technically challenging. This is one of the main reasons for sharing the architecture.

As the bus service varies by the hour of the day, the system must have the ability to autoscale to save operation cost. More resources can be allocated to handle peak loads but must be released when load reduces. As the RTTI is public facing, redundancy and error handling must be built in to handle both data errors and system failures.

Like any ML model, predictions from our models will drift over time. The system must have the ability to automatically detect when models drift beyond an acceptable threshold and trigger model retraining with more recent data. Therefore, aside from sending departure predictions to customers, a copy of the model-level predictions must also be sent to the Drift Monitoring System. With over 18,000 sets of ML models, models should have version control; model lineage information must be traceable to allow future investigations and reproduction of models if required. Models should be deployable to production independently and should not require any production system components to be redeployed. Lastly,

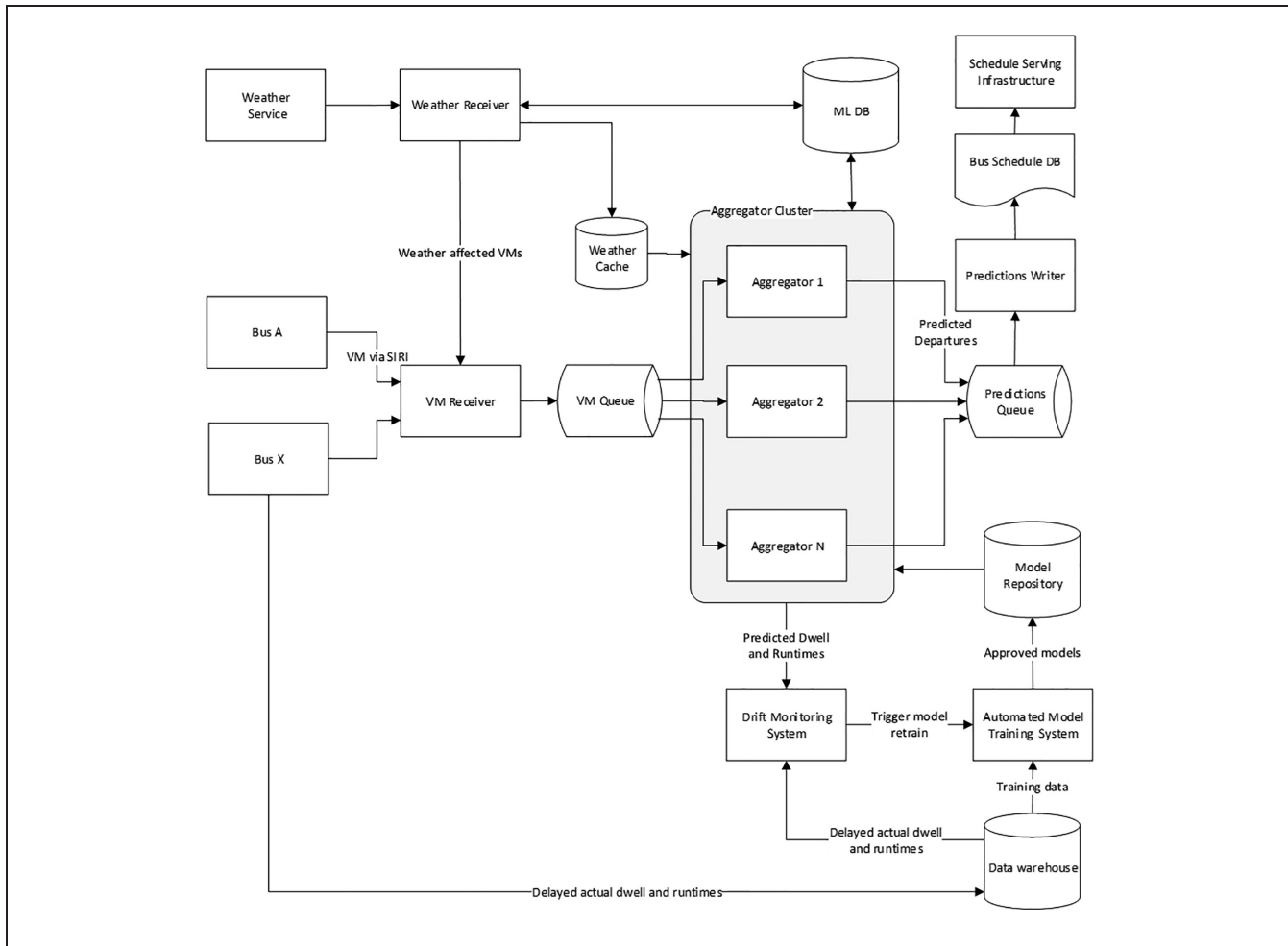


Figure 3. Architecture solution diagram.

Note: ML DB = machine learning database; VM = vehicle monitoring.

after drifted models are retrained, general statistics on the retrained models should be sent to a data scientist so they can approve the models for deployment. Models rejected by the data scientist should trigger the creation of a ticket for manual investigation.

Architecture Solution

Our implementation solution uses a queue-worker design pattern with heavy caching to achieve the previously mentioned performance and redundancy requirements (Figure 3) and meets them all.

The Vehicle Monitoring (VM) Receiver is the entry point for the VM messages, generally called AVL messages, from buses via the Service Interface for Real-time Information (SIRI) feed. It is here where messages that are sent in too frequently are filtered. Sometimes buses can send data too frequently which takes a toll on performance without adding significant value to the customers in prediction accuracy. Once filtered, the remaining

messages are sent to the VM queue, which has the added function of preventing spikes in the number of messages from overwhelming the Aggregator cluster. The cluster is made up of Aggregators that execute the Aggregator Algorithm as previously described. This is the heart of the system; all ML predictions and calculations are done by these multithreaded Aggregators. The VM queue and Aggregators form a single queue multiple worker model.

The Aggregator cluster acquires feature data required for the ML models from the machine learning database (ML DB). The cluster and the ML DB also acquire live weather data from the third-party weather service, Dark Sky API, via the Weather Receiver and the Weather Cache.

Bus prediction-level data is sent to the predictions queue, which acts as a shock absorber so that excessive numbers of predictions don't overwhelm the Bus Schedule DB. The predictions writer inserts messages in batches into the Bus Schedule DB. Whenever customers make a request for next bus times, predictions are taken from this Schedule DB.

A copy of the model-level data is sent from the Aggregator cluster to the Drift Monitoring System. All sets of models are checked for drifting by comparing model-level predictions to actual run and dwell times from the data warehouse. Models that are tested positive for drift trigger retraining for independent sets of models. They are retrained on the Automated Model Training System that acquires the latest training data from the data warehouse. Once these models are trained and scored, and meet a certain success criterion, a notification with the score report is automatically generated and sent to a data scientist for approval. Approved models are deployed to the model repository and pulled by the Aggregator cluster. Retrained models that are denied trigger a ticket creation for manual investigation. Data in the Drift Monitoring System is optimized for reporting (Star Schema) and since it contains actuals, it can also be used for retraining.

At the time of writing, TransLink has developed and fully automated the whole system except the “Drift Monitoring System” piece. We had between one to five developers at different phases of the project. If we normalized it to one senior developer, it would take approximately 33 months to develop what we have so far.

Drift Detection

While the drift data set is built, the drift model is yet to be implemented at the time of writing this paper. The algorithm to be implemented is the McDiarmid Drift Detection Method (MDDM) with the arithmetic scheme, and it was chosen for its reported fast performance and ability to minimize type I and type II errors (11). Just like our choice of the supervised ML model, with so many models we need to ensure the drift detection algorithm has a relatively short run time. The parameters for the drift method are dependent on testing results, but we have tentatively put some thought into what may be reasonable. The linear scaling factor should be relative to the window size—if the window size is V , the linear scaling factor should be $\frac{1}{V}$. The window size should be different for each segment or stop and is the number of rows of data equivalent to roughly four weeks of data.

Results

Model Performance

To measure performance of the models, we use three metrics: The mean absolute error (MAE), mean absolute percentage error (MAPE), and root mean square error (RMSE). The model metrics are then compared with the metrics of the average of the actual run or dwell times of the same stop or segment, respectively.

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |Y_i - \hat{Y}_i| \quad (5)$$

$$\text{MAPE} = \frac{1}{m} \sum_{i=1}^m \left| \frac{Y_i - \hat{Y}_i}{Y_i} \right| \times 100\% \quad (6)$$

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^m (Y_i - \hat{Y}_i)^2}{m-1}} \quad (7)$$

where Y_i is the actual run or dwell time, and $\hat{Y}_i$ is the prediction from the sets of models for that stop or segment and m is the number of observations. As buses don't have to stop at all designated bus stops during service, there are many actual dwell times that have a value of zero. Because of this, MAPE calculations are not suitable for dwell time models.

As the sets of run or dwell time models are independent, taking the average of any of the metrics wouldn't be meaningful. In Figures 4–7, we plot histograms for the 9,000 sets of run time models and 9,000 sets of dwell time models to have a better understanding of the results at the model level. A large majority of the run time models' MAPE scores improve by up to 2% compared with the historical average model. About 10% of run time models have an improvement greater than or equal to 5%. Also, a large majority of the dwell time models' MAE scores improve by up to 2 seconds. Likewise, the difference in RMSE scores for both mostly improve by up to 2 seconds. While this may seem like the method isn't effective, it is expected that there are many stops and segments where the run or dwell time would be very static. For example, very infrequent stops would consistently have a dwell time of zero. Segments in low density areas that are short and don't cross intersections would have a very consistent run time as they would be insensitive to time of day, day type, or weather.

However, as our departure predictions are composed of multiple dwell and run time predictions, it only takes one significantly improved run or dwell prediction giving a relatively large improvement in the overall prediction to outperform the historical average. This is especially true given that an error in the predicted time of one bus stop affects the predicted time of all subsequent stops. To visualize the segments and stops that have ML predictions with the largest relative improvements over the average, we can group the differences and plot them on a map using different colors as in Figure 8.

The segments that have an ML run time MAPE difference of 5% or higher than the historical average model are located in various places throughout the map and are not centralized to a few locations. Many of these segments are in areas that are susceptible to unexpected congestion or are near frequently busy intersections. With run time models that have a large improvement in various areas, most bus lines in our system would have at

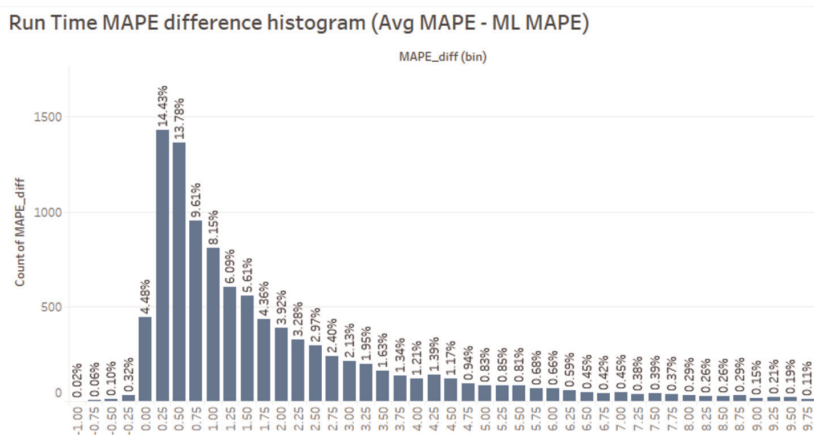


Figure 4. Histogram of the differences in mean absolute percentage error for run time models.

Note: ML = machine learning; MAPE = mean absolute percentage error.

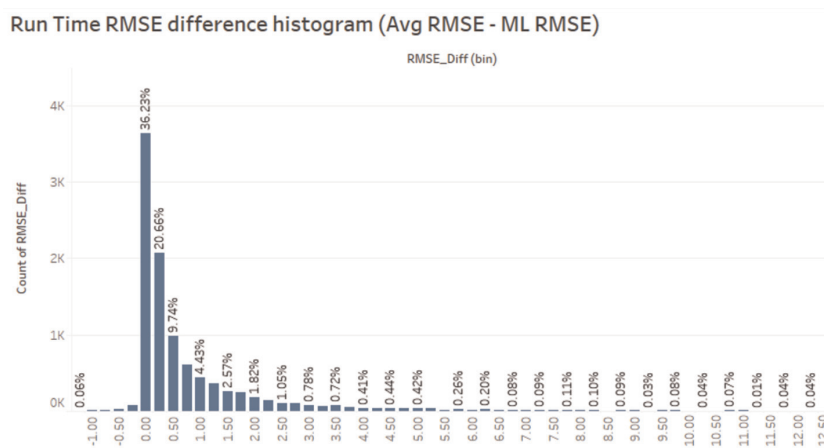


Figure 5. Histogram of the differences in root mean square error for run time models.

Note: ML = machine learning; RMSE = root mean square error.

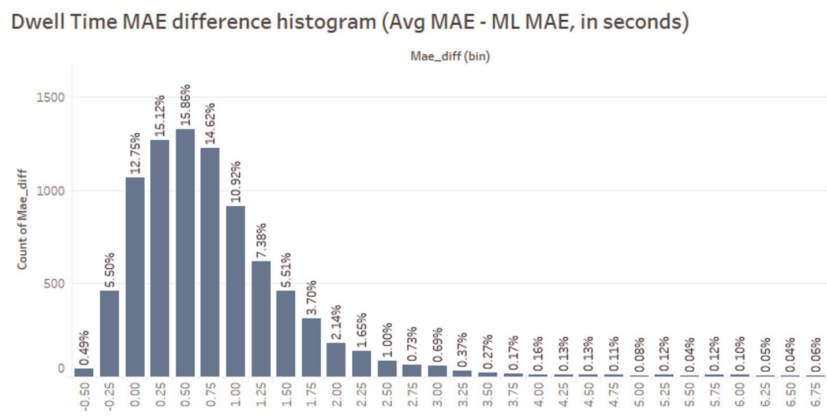


Figure 6. Histogram of the differences in mean absolute error for dwell time models.

Note: ML = machine learning; MAE = mean absolute error.

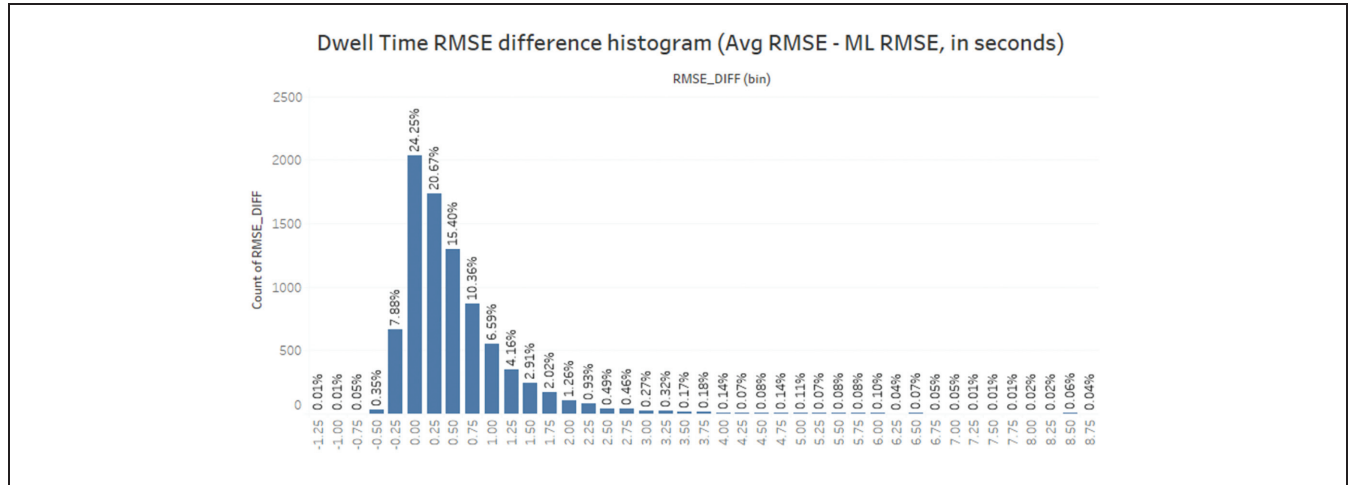


Figure 7. Histogram of the differences in mean absolute error for dwell time models.

Note: ML = machine learning; RMSE = root mean square error. All metrics in Figures 4–7 are binned by 0.25. Example: 0 = [0,0.25), 0.25 = [0.25,0.50). All graphs are right-skewed, and most of the tail is not shown.

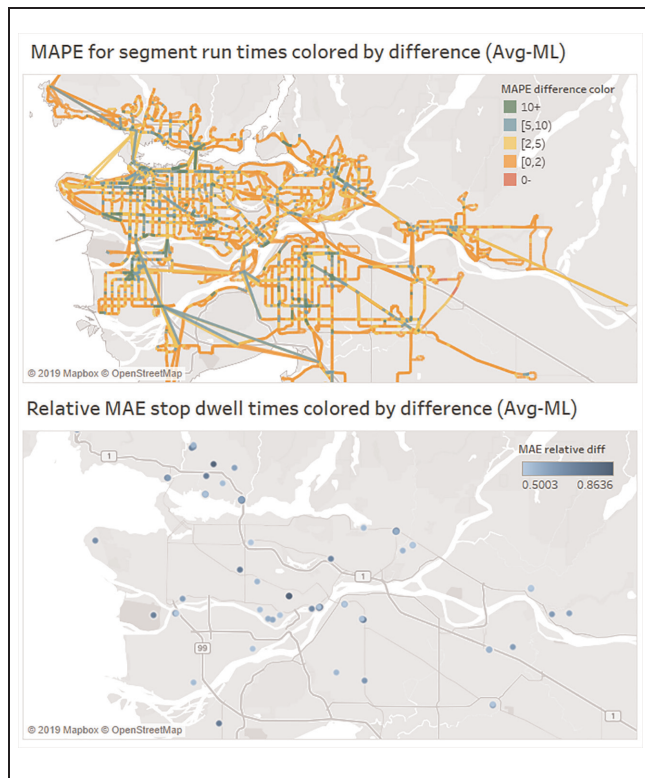


Figure 8. Segment run time difference by color.

Note: ML = machine learning; MAE = mean absolute error; MAPE = mean absolute percentage error. In the top diagram, the segments don't follow the exact route path but are representative lines drawn between two stops. In the bottom diagram, only stops with a relative mean absolute error difference of 50% or higher are shown.

least one of these segments along their route, which would mean a significant improvement at the prediction level. The stop dwell time graph is colored by the relative

MAE differences, which are calculated as $\frac{MAE_{avg} - MAE_{ML}}{MAE_{avg}}$. Interestingly, we see that majority of stops that have the largest dwell time prediction improvements (only stops with large improvements are shown) are stops at bus loops or exchanges. These stops are usually the first or last stop in a service route, and many of these stops are close to SkyTrain stations.

At the model level, we see that 99% of all our models are better than the historical average model (those that are below require additional training time). We also see visually that most service routes have at least one stop or segment where the predictions are significantly improved by ML. The models with large relative improvements are likely a result of extreme conditions that the average model can't account for (e.g., conditions caused by traffic congestion resulting from extreme weather conditions). Extreme conditions are situations when customers are most affected, and when predictions are off by more than 5 min, customers' perceptions of wait times are exaggerated and these occasions become very memorable (12).

Overall Actual System Performance by Horizon

To measure the overall performance of the completed system, with ML and Aggregator Algorithm integrated, we use two metrics: The absolute error percentiles by Horizon and average absolute error with 95% confidence intervals by Horizon. Both metrics are based on the same data set. This data set was collected for all routes in the TransLink bus network from September 1, 2019 to February 29, 2020. The records were generated by random samples of customers asking for the next three buses for a stop. Therefore, up to three records are generated for each ask. Horizon is the duration between when a

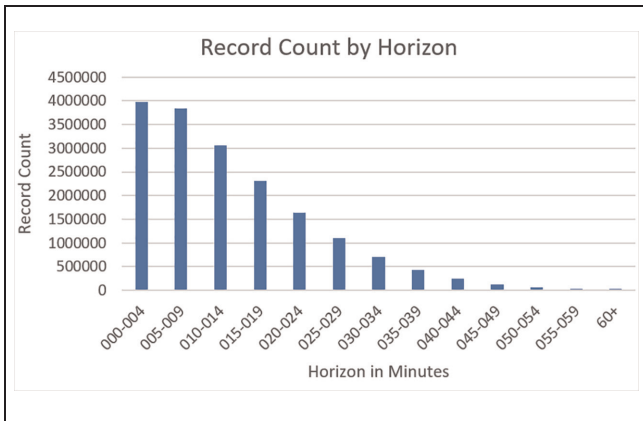


Figure 9. Histogram of record count by Horizon.

customer asked for a prediction and the actual bus departure time for the stop. Horizons are then grouped into bins to simplify analytics. For example, 005–009 means the bus is 5–9 min from actual departure.

Figure 9 shows the distribution of records across Horizon. It shows that most bus routes in the TransLink network have frequent service. The next three buses are rarely beyond the next hour.

Figure 10 shows the distribution of absolute errors, defined by Equation 8, by Horizon. As expected, the further away the bus, the higher the uncertainty, and therefore the higher the absolute error. For example, when the bus is 0–4 min away, the median is 26 s. When the bus is 30–34 min away, the median increases to 125 s.

$$\text{Absolute Error} = |\text{Actual Departure Time} - \text{ML Predicted Departure Time}| \quad (8)$$

With our algorithm we expect the error to increase linearly, like 000–004 to 030–034, and not exponentially like

040–044 onwards. We investigated this issue and found that the unexpected increases are mostly field operations related. The further that we predict into the future the higher the probability of having nodes in the projected path. These nodes are timing points and can also be points where drivers start or end their shifts. We found that 12% of the time, bus operators don't adhere to timing point rules, where if they're early, they are supposed to wait until the scheduled departure time before departing. When this happens, predictions for all stops after the timing point will be skewed until the bus eventually departs the timing point. We are using ML to shave error by the second and these operation issues can easily skew the results by many minutes. Even worse, some bus drivers are late for their shift. Now that we've identified some of these issues, it may help to add bus driver statistics as inputs to our models.

Figure 11 shows the average absolute errors with 95% lower and upper confidence intervals by Horizon. The farther away the bus, the wider the confidence interval but overall, it's quite tight throughout. Table 3 shows the details of three sample data points.

Comparison with RTTI (Historical Average)

Here, we offer a comparison with RTTI because this is likely the real-time departure estimate method a transit agency will adopt if it uses popular software systems such as "Trapeze" for creating its bus schedules. To understand RTTI departure time predictions, we must first understand how the planned bus schedule is created.

In TransLink, as shown in Figure 12, routes contain both nodes and stops. Nodes are special stops used for timing purposes and include the start stop and the terminus stop. A scheduler only provides the estimated

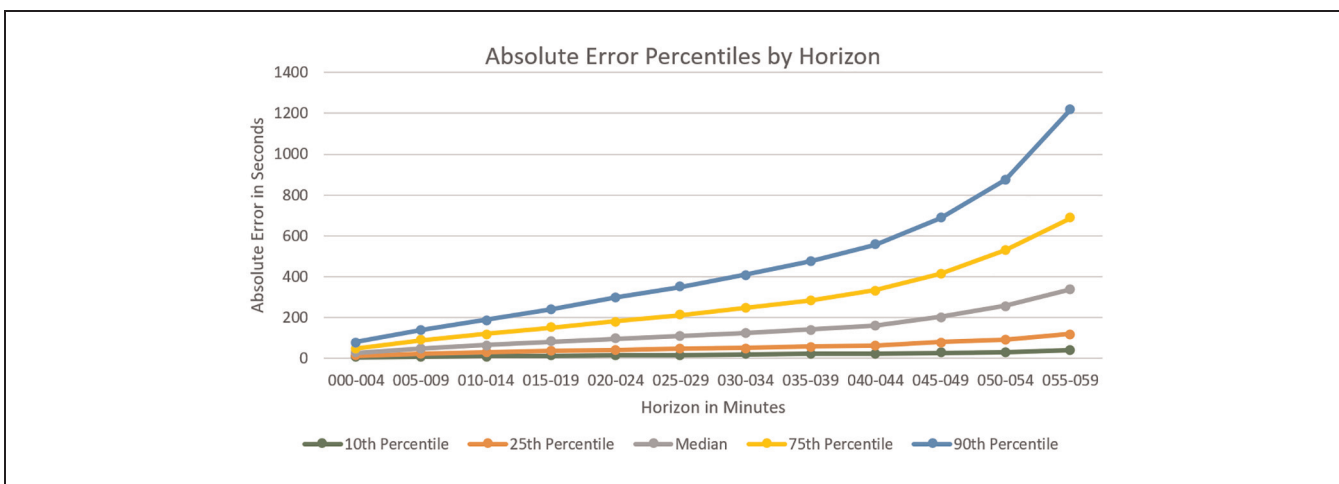


Figure 10. Absolute error percentiles by Horizon.

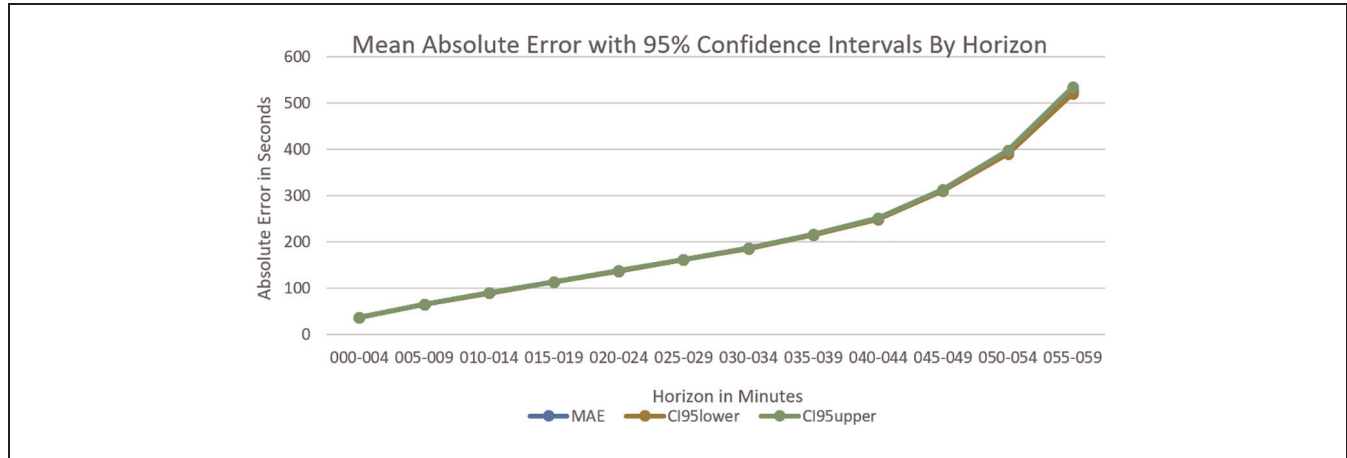


Figure 11. Mean absolute error (MAE) with 95% confidence interval (CI) by Horizon.

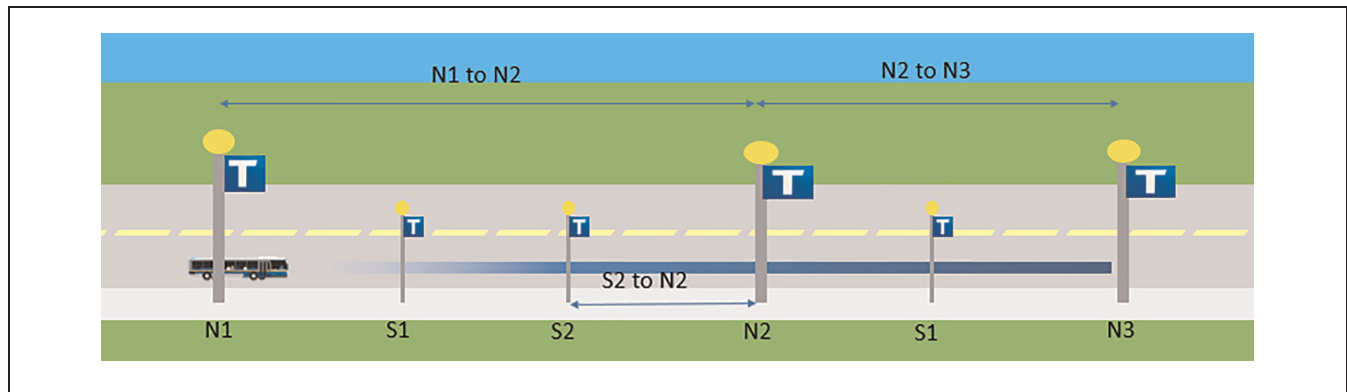


Figure 12. Route with nodes and stops.

Table 3. Sample Points for Mean Absolute Error (MAE) with Confidence Interval (CI)

Horizon (min)	MAE (s)	95% CI lower (s)	95% CI upper (s)
000–004	37	36.94	37.06
030–034	187	186.49	187.51
055–059	528	520.62	535.38

duration between nodes and the system automatically computes the duration between stops by prorating the total node-to-node duration by distance. For example, if N1 to N2 is estimated to be 300 s and has a total distance of 450 m, and S2 to N2 is 150 m, then S2 to N2 would be assigned a duration of 100 s ($150/450 \times 300$ s). Node-to-node duration is the previous year's average by day of week and hour of day. For example, if a trip is scheduled to leave N1 at 4:15 p.m. on a Friday then it will be assigned the average duration for 4–5 p.m. on weekdays.

Now that we know how the planned schedule is created, pre-ML RTTI predictions are simply real-time schedule shifts. If the AVL on a bus says that it's 5 min behind schedule, RTTI will report scheduled departure times for all subsequent stops with 5 min added to them.

Like the Horizon analysis, the following analysis is done on the entire TransLink bus network from September 01, 2019 to February 29, 2020. Operation issues that we have not been able to isolate also exist in this data set so the results should be interpreted as a below average if not a worst case scenario.

From Figure 13, the total number of ML predictions where the bus is “on-time” is higher than RTTI predictions by about 12% of all predictions. Predictions where it is off by more than 5 min dropped by almost 2%. The prediction-level MAE for RTTI is 126 s, while it is 96 s for ML predictions. This is a difference of 23.8% prediction-level MAE. Next, we look at a more customer-based metric—the average unexpected wait time per boarding. For a particular stop for the i th bus, the total unexpected wait time of all boardings, U_i , is calculated as

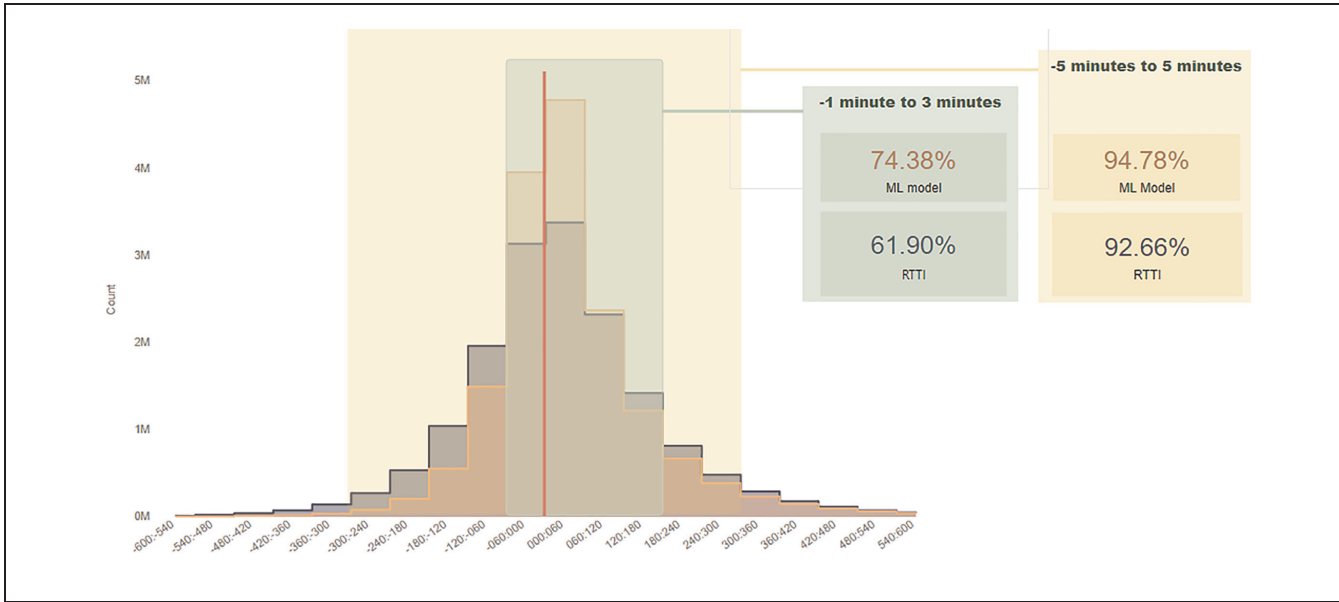


Figure 13. Prediction-level bus departure time comparisons: actual versus machine learning (orange) and actual versus real time transit information (blue). A negative value means that the actual bus departure time was earlier than the predicted time. The green box represents predictions where the bus was “on-time” according to our transit standards—the bus left at most 1 min earlier or at most 3 min later than the prediction. The yellow box represents predictions that are within 5 min of true bus departure times.

Note: ML = machine learning; RTTI = Real Time Transit Information.

$$U_i = \begin{cases} g_{(i+1)} \times \frac{p_i - t_i}{t_{(i+1)} - t_i} \times (t_{(i+1)} - p_i) & \text{if } p_i > t_i \\ g_i \times (t_i - p_i) & \text{if } p_i \leq t_i \end{cases} \quad (9)$$

where

t_i is the time of the i th bus departing;

g_i is the number of known boardings at that stop; and

p_i is the predicted departure time.

This calculation assumes the customers arrive at the predicted departure time. If the bus departs earlier than the predicted time, it takes the estimated number of customers that would have waited since the previous prediction time $g_{(i+1)} \times \frac{p_i - t_i}{t_{(i+1)} - t_i}$ and the time until the next bus departs $t_{(i+1)} - p_i$ into account. If the bus departs later than our prediction, it is simply the total number of customers times the difference in departure times. The average unexpected wait time per boarding is the sum of all unexpected wait times divided by the total number of boardings.

The RTTI predictions had an average unexpected wait time per boarding of 151 s, while it was 125 s for the ML predictions. This means that, on average, customers spend 17% less time on unexpected waits for buses with our new ML predictions.

Conclusion and Future Work

We showed that by training a model for each segment and stop, we can achieve very reliable predictions for our customers even without key data sets like traffic and

service demand data. In addition, we presented an algorithm that combines these predictions efficiently while also handling business rules and late arriving training data for added or removed stops. We also shared an enterprise grade architecture that is scalable and cost effective. The architecture fully automates the detection of drifting models and retraining, which makes the system self-learning.

Aggregating various dwell and run time predictions to acquire a departure time prediction is a flexible method that is insensitive to changes in bus stop, bus line, and schedule. The method requires training of thousands of models for most bus systems, and, because of this, careful consideration of how to implement and handle these models is required. The Aggregator Algorithm needs to be tweaked for the business rules behind the bus service data, and the implementation architecture needs to be well-structured and robust.

At the model level, the results are shown to be better for almost all segments and bus stops, providing a relatively large improvement for some ML models when compared with the historical average as measured by MAPE, MAE, and RMSE. Visually, it is shown that most service paths have at least a few models in which the run or dwell time predictions are dramatically improved. The dwell time models that improved the most are ones that are at bus loops.

At the PBDT level, we analyzed the error or drift by Horizon and showed that, as expected, the farther away

the bus, the larger the error. The analysis also showed that the predictions are reliable even for a bus that is up to an hour away. Beyond that, the chance of nonadherence to operation rules increases, resulting in wider errors.

Also, at the PBDT level, analysis was done to compare the ML bus departure time predictions to existing RTTI historical average predictions. The ML predictions outperform the original RTTI predictions. Out of all bus stop events, the ML predictions have 12% more “on-time” predictions.

A method to estimate the average unexpected wait time per boarding is also discussed. Compared with the RTTI predictions, the ML predicted departure time reduces the average unexpected wait time per boarding by 17%.

Other non-XGBoost ML methods should also be tested. Although many other ML methods may require more training time, the accuracy could be improved, and it would be useful to understand the impact at the prediction level. It would be very interesting to see if additional features such as bus driver, special events, and congestion would affect the results.

TransLink’s XGBoost models were trained on the previous 2 years of historical data. It would be interesting to see if training on just recent data (e.g., just the previous month’s) would produce better predictions. On the flip side, it would also be interesting to see the impact of including older historical data to increase the size of the training data set.

TransLink buses are configured to send location messages to the central system whenever it leaves a stop or every 90 s, whichever is sooner. On the ML receiving end, we have a filter to discard messages that are sent in too soon after the last message. Our reports showed that increasing the filter from 30 to 90 s only decreased PBDT accuracy by 1%, while reducing ML compute costs by 50% per month. We concluded that if the ML algorithm is accurate, there is no need to compute too frequently. Companies should conduct their own sensitivity analysis to find the right balance.

Acknowledgments

This project, and therefore this paper, was made possible by TransLink. We thank T4G Limited for their services, which include feature engineering and creation of the pilot version of the model sets. We also thank Microsoft Corporation for sponsoring the proof-of-concept (POC) phase of this project. This spawned further interest in the project.

Author Contributions

The authors confirm contribution to the paper as follows: study conception and design: Winston Zhou, Benny Wai; data collection: Winston Zhou, Benny Wai; analysis and interpretation of

results: Winston Zhou, Benny Wai; draft manuscript preparation: Winston Zhou, Benny Wai. All authors reviewed the results and approved the final version of the manuscript.

Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) received no financial support for the research, authorship, and/or publication of this article.

References

1. Amita, J., J. S. Singh, and G. P. Kumar. Prediction of Bus Travel Time using Artificial Neural Network. *International Journal for Traffic and Transport Engineering*, Vol. 5, No. 4, 2015, pp. 410–424.
2. Fan, W., and Z. Gurmu. Dynamic Travel Time Prediction Models for Buses using Only GPS Data. *International Journal of Transportation Science and Technology*, Vol. 4, No. 4, 2015, pp. 353–366.
3. Kumar, V., B. A. Kumar, L. D. Vanajakshi, and S. C. Subramanian. Comparison of Model Based and Machine Learning Approaches for Bus Arrival Time Prediction. Presented at 93rd Annual Meeting of the Transportation Research Board, Washington, D.C., 2014.
4. Yin, T., G. Zhong, J. Zhang, S. He, and B. Ran. A Prediction Model of Bus Arrival Time at Stops with Multi-Routes. *Transportation Research Procedia*, Vol. 25, 2017, pp. 4623–4636.
5. Patnaik, J., S. Chien, and A. Bladikas. Estimation of Bus Arrival Times using APC Data. *Journal of Public Transportation*, Vol. 7, No. 1, 2004, pp. 1–20.
6. Shalaby, A., and A. Farhan. Prediction Model of Bus Arrival and Departure Times using AVL and APC Data. *Journal of Public Transportation*, Vol. 7, No. 1, 2014, pp. 41–61.
7. Peterson, N. C., F. Rodrigues, and F. C. Pereira. Multi-Output Deep Learning for Bus Arrival Time Predictions. *Transportation Research Procedia*, Vol. 41, 2018, pp. 138–145.
8. Dark Sky API. <https://darksky.net/dev>. Accessed June 25, 2019.
9. Distributed (Deep) Machine Learning Community. <https://github.com/dmlc/xgboost/tree/master/demo#machine-learning-challenge-winning-solutions>. Accessed May 5, 2020.
10. Cambridge Spark. <https://blog.cambridgespark.com/hyperparameter-tuning-in-xgboost-4ff9100a3b2f>. Accessed May 5, 2020.
11. Pesaranghader, A., H. L. Viktor, and E. Paquet. McDiarmid Drift Detection Methods for Evolving Data Streams. *Proc., 2018 International Joint Conference on Neural Networks*, Rio de Janeiro, Brazil, IEEE, New York, 2018.
12. Fan, Y., A. Guthrie, and D. Levinson. Waiting Time Perceptions at Transit Stops and Stations: Effects of Basic Amenities, Gender, and Security. *Transportation Research Part A: Policy and Practice*, Vol. 88, 2016, pp. 251–264.